Université de Rouen Normandie

L3 Informatique – Parcours Informatique

Projet AlgoGraphes

Tournées Hamiltoniennes de Coût Minimum

Algorithmes d'Approximation et Branch & Bound

Étudiants :
Anis AISSANI
Saadallah Oussama KHELLIFA

Encadrant :
O. Mallet

Année universitaire 2025-2026

Table des matières

1 Introduction **2**

2 Structures de Données Algorithmiques **2**
2.1 GrapheMD : Matrice des Distances 2
2.2 GrapheTL : Tableau de Listes d'Adjacence 2
2.3 Tas : File de Priorité (Min-Heap) 3
2.4 NoeudExploration : État pour Branch & Bound 4

3 Algorithme du Point Plus Proche (PPP) **4**
3.1 Procédure Détaillée 5
3.2 Implémentation Python 5

4 Optimisation PPP (OptPPP / 2-Opt) **7**
4.1 Procédure Détaillée 8
4.2 Implémentation Python 8

5 Approximation par Arbre Couvrant (PvcPrim) **10**
5.1 Procédures Détaillées 10
5.2 Implémentation Python 11

6 Heuristique de la Demi-Somme (HDS) **14**
6.1 Procédures Détaillées 15
6.2 Implémentation Python 16

7 Bilan du Projet : Organisation et Analyse Critique **20**
7.1 Organisation du Travail 20
7.2 Difficultés Rencontrées 21
7.3 Idées d'Amélioration 21
7.4 Analyse de Complexité des Algorithmes 21

8 Étude Statistique et Résultats **22**
8.1 Performance et Robustesse (N=10) 22
8.2 Évolution de la Complexité (N variable) 23
8.3 Démonstration Visuelle (Cas N=18) 24

9 Conclusions **26**

10 Références **27**
10.1 Cours et Documentation Pédagogique 27
10.2 Ressources Algorithmiques 27
10.3 Bibliothèques Python 27

1 Introduction

Le problème du voyageur de commerce (TSP) consiste à trouver le cycle hamiltonien de coût minimal dans un graphe complet pondéré. Ce rapport présente l'implémentation et l'analyse détaillée de quatre algorithmes : PPP, OptPPP, OptPrim et HDS.

2 Structures de Données Algorithmiques

Pour garantir l'efficacité et respecter les consignes du cours, nous avons implémenté quatre structures de données spécifiques.

2.1 GrapheMD : Matrice des Distances

Description : Cette structure contient le nombre de sommets n et une matrice carrée D où $D[i][j]$ stocke la distance euclidienne entre le sommet i et le sommet j.

Justification : Le problème du voyageur de commerce porte sur un graphe complet (n^2 arêtes). La matrice est la structure la plus efficace dans ce contexte car elle permet d'accéder à la distance entre deux villes n'importe lesquelles en temps constant $O(1)$. Cette complexité est cruciale pour ne pas ralentir les algorithmes PPP et OptPPP qui effectuent de nombreuses comparaisons de distances.

```python
# (Classe GrapheMD)
class GrapheMD:
    def __init__(self , n , points ) :
        self.n = n
        self.points = points
        self.D = np.zeros((n,n))
        self._calculer_distance_euclidienne()


    def _calculer_distance_euclidienne(self):

        for i in range(self.n):
            for j in range(self.n):
                if i != j:
                    xi , yi = self.points[i]
                    xj , yj = self.points[j]
                    self.D[i][j] = np.sqrt((xi - xj)**2 + (yi -
                        yj)**2)
                else:
                    self.D[i][j] = 0.0
```

Listing 1 – Code source : graphe_md.py

2.2 GrapheTL : Tableau de Listes d'Adjacence

Description : Il s'agit d'un tableau où la case i contient la liste chaînée (ou dynamique) des voisins du sommet i.

Justification : Cette structure est utilisée spécifiquement pour l'algorithme OptPrim. L'algorithme de Prim construit un Arbre Couvrant Minimum (MST), qui est un graphe creux contenant beaucoup moins d'arêtes qu'un graphe complet ($n-1$ arêtes contre n^2). Stocker un tel arbre dans une matrice serait un gaspillage significatif de mémoire et de temps de parcours.

Détail Technique : Notre implémentation permet d'ajouter des arêtes non-orientées (pour la définition mathématique du MST) et des arcs orientés (pour faciliter le parcours préfixe demandé par le sujet).

```python
# (Classe GrapheTL)
import numpy as np
import heapq # For priority queue implementation


# --- STRUCTURE 2 :  GrapheTL (Tableau de Listes d'Adjacence) ---

class GrapheTL:
    def __init__(self, n):
        self.n = n
        self.T = [[] for _ in range(n)]  # Initialisation du tableau de listes d'adjacence

    def ajouter_arete(self, u, v, poids):
        self.T[u].append((v, poids))
        self.T[v].append((u, poids))

    def ajouter_arc(self, u, v, poids):
        self.T[u].append((v, poids))
```

Listing 2 – Code source : graphe_tl.py

2.3 Tas : File de Priorité (Min-Heap)

Description : C'est un arbre binaire stocké sous forme de tableau (ou liste), permettant d'accéder à l'élément minimum en temps $O(1)$ et d'insérer un nouvel élément en $O(\log n)$.

Justification : Le sujet demande la "version la plus efficace" de Prim et une "recherche intelligente" pour HDS. Ces deux algorithmes nécessitent d'extraire rapidement le meilleur candidat à chaque étape : le sommet le plus proche pour Prim, ou la solution partielle la plus prometteuse (plus petite borne inférieure) pour HDS.

```python

# (Classe Tas)
class Tas:
    def __init__(self):
        self.elements = []  # Initialisation de la liste des éléments du tas

    def inserer(self, element , priority):
```

```python

        heapq.heappush(self.elements, (priority, element))

    def extraire_min(self):
        return heapq.heappop(self.elements)

    def est_vide(self):
        return len(self.elements) == 0
```

Listing 3 – Code source : tas.py

2.4 NoeudExploration : État pour Branch & Bound

Description : C'est un enregistrement (classe) qui représente une "solution partielle" dans l'arborescence de recherche de l'algorithme Branch & Bound.

Contenu : Il stocke la ville actuelle, un masque binaire des villes déjà visitées, le coût actuel du chemin parcouru $g(x)$ et l'estimation heuristique $h(x)$ (basée sur la Demi-somme).

```python
# (Classe NoeudExploration)
class NoeudExploration:
    def __init__(self, current_city, visited_mask, cost, bound, path):
        self.current_city = current_city # Sommet actuel
        self.visited_mask = visited_mask # Masque des visités
        self.cost = cost                 # Coût g(x)
        self.bound = bound               # Heuristique h(x)
        self.path = path                 # Chemin partiel

    def __lt__(self, other):
        # Comparaison pour le Tas : on priorise la plus petite borne
        return self.bound < other.bound
```

Listing 4 – Code source : noeud_exploration.py

3 Algorithme du Point Plus Proche (PPP)

L'algorithme PPP est une heuristique constructive gloutonne. Il construit le cycle en ajoutant à chaque étape la ville non visitée la plus proche du cycle actuel, en choisissant l'insertion (gauche ou droite) la plus économique.

3.1 Procédure Détaillée

Algorithm 1 Algorithme PPP (Point le Plus Proche)

Entrée: Graphe complet G avec matrice des distances D
Sortie: Cycle Hamiltonien C

```
▷ 1. Initialisation
n ← G.n
C ← [0]                                   ▷ On commence par la ville 0
NonVisites ← {1, 2, ..., n − 1}
▷ Trouver le plus proche voisin du point de départ
v_proche ← argmin_{v∈NonVisites}(D[0][v])
Ajouter v_proche à C
Retirer v_proche de NonVisites
▷ 2. Boucle Principale
tant que NonVisites ≠ ∅ faire
  ▷ Étape A : Sélection du couple (Qi, Qj) minimal
  MinDist ← ∞
  Qi ← −1, Qj ← −1, IndexQj ← −1
  pour chaque u ∈ NonVisites faire
    pour chaque v ∈ C à la position k faire
      si D[u][v] < MinDist alors
        MinDist ← D[u][v]
        Qi ← u, Qj ← v, IndexQj ← k
      end si
    end pour
  end pour
  ▷ Étape B : Calcul des coûts d'insertion
  Gauche ← C[(IndexQj − 1)%|C|]
  Droite ← C[(IndexQj + 1)%|C|]
  CoutG ← D[Gauche][Qi] + D[Qi][Qj] − D[Gauche][Qj]
  CoutD ← D[Qj][Qi] + D[Qi][Droite] − D[Qj][Droite]
  ▷ Étape C : Insertion à la meilleure position
  si CoutG < CoutD alors
    Insérer Qi à la position IndexQj (à gauche de Qj)
  sinon
    Insérer Qi à la position IndexQj + 1 (à droite de Qj)
  end si
  Retirer Qi de NonVisites
end tant que
retourner C
```

3.2 Implémentation Python

```python
from structures.graphe_md import GrapheMD

```

```python

# Algorithme du Point Plus Proche (PPP)

def algo_ppp(G):
    n = G.n
    D = G.D

    # Cas particulier : si le graphe est vide ou contient un seul point
    if n == 0:
        return []
    if n == 1:
        return [0]

    # --- 1. Initialisation du cycle avec le point 0 et son plus proche voisin
    cycle = [0]
    non_visites = set(range(1, n))

    # Trouver le plus proche voisin du point 0
    plus_proche_voisin = min(non_visites, key=lambda x: D[0][x])
    cycle.append(plus_proche_voisin)
    non_visites.remove(plus_proche_voisin)

    # --- 2. Boucle principale de l'algorithme PPP
    # Inserer les points restants dans le cycle
    while non_visites:
        # 1. Etape 1 : La selection
        # Trouver la ville Qi la plus proche du cycle courant
        # On cherche le couple (Qi , Qj) tel que :
        #     - Qi appartient a non_visites
        #     - Qj appartient a cycle
        #     - La distance D(Qi , Qj) est minimale
        min_distance = float('inf')
        Qi = -1
        Qj = -1
        for u in non_visites:
            for idx , v in enumerate(cycle):
                dist = D[u][v]
                if dist < min_distance :
                    min_distance = dist
                    Qi = u
                    Qj = v
                    Qj_index = idx

        # 2. Etape 2 : L'insertion
        # Inserer Qi dans le cycle C juste a cote de Qj
        # Soit a gauche ou a droite de Qj
```

```
        # En choisissant la position qui minimise l'augmentation de la longueur totale du cycle C

        # Calculer le cout d'insertion a gauche de Qj
        # Identifier le voisin gauche de Qj dans le cycle
        Qj_gauche_index = (Qj_index - 1) % len(cycle)
        Qj_droite_index = (Qj_index + 1) % len(cycle)

        Qj_gauche = cycle[Qj_gauche_index]
        Qj_droite = cycle[Qj_droite_index]

        # Cout d'insertion a gauche
        cout_gauche = D[Qj_gauche][Qi] + D[Qi][Qj] - D[Qj_gauche][Qj]
        # Cout d'insertion a droite
        cout_droite = D[Qj][Qi] + D[Qi][Qj_droite] - D[Qj][Qj_droite]

        # Choisir la position qui minimise le cout
        if cout_gauche < cout_droite:
            # Insérer Qi à gauche de Qj
            cycle.insert(Qj_index, Qi)
        else:
            # Insérer Qi à droite de Qj
            cycle.insert(Qj_droite_index, Qi)

        non_visites.remove(Qi)

    return cycle
```

Listing 5 – Code source : algo_ppp.py

4 Optimisation PPP (OptPPP / 2-Opt)

OptPPP est une heuristique d'amélioration locale. Elle parcourt le cycle pour détecter des croisements d'arêtes. Si le remplacement de deux arêtes (A, B) et (C, D) par (A, C) et (B, D) réduit la distance totale, une opération de "décroisement" (inversion de segment) est effectuée.

4.1 Procédure Détaillée

Algorithm 2 Algorithme OptPPP (Optimisation 2-Opt)

Entrée: Cycle initial C_{init}, Graphe G (Matrice D)
Sortie: Cycle optimisé C

$C \leftarrow C_{init}.copie()$
$n \leftarrow longueur(C)$
$Amelioration \leftarrow Vrai$
tant que $Amelioration$ **faire**
  $Amelioration \leftarrow Faux$
  ▷ *On teste toutes les paires d'arêtes non adjacentes*
  pour i de 0 à $n-1$ **faire**
    pour j de $i+2$ à $n-1$ **faire**
      ▷ *Éviter le cas où la dernière arête boucle sur la première*
      si $i == 0$ ET $j == n-1$ **alors**
        Continuer
      end si
      $A \leftarrow C[i], B \leftarrow C[(i+1)\%n]$
      $C_{pt} \leftarrow C[j], D_{pt} \leftarrow C[(j+1)\%n]$
      ▷ *Comparaison des distances*
      $Dist_{actuelle} \leftarrow D[A][B] + D[C_{pt}][D_{pt}]$
      $Dist_{nouvelle} \leftarrow D[A][C_{pt}] + D[B][D_{pt}]$
      si $Dist_{nouvelle} < Dist_{actuelle}$ **alors**
        ▷ *On inverse le segment entre B et C pour décroiser*
        $C[i+1:j+1] \leftarrow Inverser(C[i+1:j+1])$
        $Amelioration \leftarrow Vrai$
      end si
    end pour
  end pour
end tant que
retourner C

4.2 Implémentation Python

```python
from structures.graphe_md import GrapheMD

# Version optimisée de l'algorithme du Point Plus Proche (OptPPP)

def opt_ppp(cycle_init, G):
    D = G.D

    # Copie du cycle initial , pour ne pas le modifier directement
    cycle = cycle_init.copy()
    n = len(cycle)

    amelioration = True
```

```python

    # Boucle principale de l'optimisation
    while amelioration:
        amelioration = False
        # Parcourir toutes les paires d'arêtes (i, i+1)
        for i in range(n):
            # on la compare avec toutes les arêtes (j, j+1) suivantes
            # j commence a i+2 pour eviter les arêtes adjacentes
            for j in range(i + 2, n):
                # Cas particulier : on ne compare pas la derniere arête avec la premiere
                # car elles sont adjacentes dans le cycle
                # donc si j == n-1 et i == 0 , on saute cette itération
                if i == 0 and j == n - 1:
                    continue

                # Indices des points dans le cycle
                A = cycle[i]
                B = cycle[(i + 1) % n]
                C = cycle[j]
                D = cycle[(j + 1) % n]

                # 1. Calculer les distances

                # Calculer la distance actuelle des arêtes (A,B) et (C,D)
                dist_actuelle = G.D[A][B] + G.D[C][D]

                # Calculer la distance si on remplace par (A,C) et (B,D)
                dist_nouvelle = G.D[A][C] + G.D[B][D]

                # 2. Vérifier si le remplacement réduit la distance totale
                if dist_nouvelle < dist_actuelle:
                    # Il y a une amélioration, on effectue le remplacement
                    # Pour cela, on inverse la section du cycle entre B et C
                    cycle[i + 1:j + 1] = reversed(cycle[i + 1:j + 1])
                    amelioration = True

    return cycle
```

Listing 6 – Code source : opt_ppp.py

5 Approximation par Arbre Couvrant (PvcPrim)

L'algorithme PvcPrim utilise la propriété structurelle des arbres couvrant minimum (MST) pour garantir une 2-approximation (sous inégalité triangulaire). Il se décompose en trois sous-procédures distinctes : Prim, Conversion, et DFS.

5.1 Procédures Détaillées

Algorithm 3 Algorithme Principal : PvcPrim

Entrée: Graphe G
Sortie: Cycle Hamiltonien

```
▷ 1. Construction du MST (retourne tableau de pères)
π ← Prim(G)
▷ 2. Transformation du tableau π en liste d'adjacence
Arbre ← Pi_Vers_GrapheTL(π)
▷ 3. Parcours Préfixe pour obtenir le cycle
Cycle ← DFS(Arbre)
retourner Cycle
```

Algorithm 4 Sous-procédure : Prim (avec Tas Binaire)

Entrée: Graphe G
Sortie: Tableau des prédécesseurs π

```
Cl[0...n − 1] ← ∞, Cl[0] ← 0
π[0...n − 1] ← −1
Visit[0...n − 1] ← Faux
F ← Tas(), F.inserer(0, 0)                    ▷ Tas contenant (sommet, priorité)
tant que NON F.est_vide() faire
    u ← F.extraire_min()
    si Visit[u] alors
        Continuer
    end si
    Visit[u] ← Vrai
    pour chaque v dans G faire
        si NON Visit[v] ET G.D[u][v] < Cl[v] alors
            Cl[v] ← G.D[u][v]
            π[v] ← u                          ▷ u devient le parent de v
            F.inserer(v, Cl[v])
        end si
    end pour
end tant que
retourner π
```

Algorithm 5 Sous-procédure : DFS (Parcours en Profondeur)

Entrée: Graphe (Arbre) G_{TL}
Sortie: Liste P^* (Ordre de découverte)

$Couleur[0...n-1] \leftarrow BLANC$
$P^* \leftarrow []$
Fonction Visiter_Recursif(u) :
$Couleur[u] \leftarrow GRIS$
Ajouter u à P^* ▷ *Enregistrement préfixe*
$Voisins \leftarrow G_{TL}.get_voisins(u)$
pour chaque v dans $Voisins$ **faire**
  si $Couleur[v] == BLANC$ **alors**
    Visiter_Recursif(v)
  end si
end pour
$Couleur[u] \leftarrow NOIR$
Fin Fonction
pour i de 0 à $n-1$ **faire**
  si $Couleur[i] == BLANC$ **alors**
    Visiter_Recursif(i)
  end si
end pour
retourner P^*

5.2 Implémentation Python

```python
# Algorithme d'approximation par Prim pour le problème du voyageur de commerce (TSP)
def opt_prim(G):

    # 1. Construire l'arbre couvrant de poids minimal (MST)
    pi = prim(G)

    # 2. Convertir pi en GrapheTL
    arbre_mst = pi_vers_graphe_tl(pi)

    # 3. Effectuer un parcours en profondeur (DFS) de l'arbre pour obtenir un cycle hamiltonien approximatif
    P , S , P_star , S_star , pi_dfs = dfs(arbre_mst)

    # P_star contient le cycle hamiltonien approximatif
    return P_star
```

Listing 7 – Code source : opt_prim.py

```python
# --- 4. DFS
# Augmentation de la limite de récursion pour les grands graphes
sys.setrecursionlimit(200000)
```

```python

def dfs(GTL) :

    n = GTL.n

    # Initialisations
    ip = 0  # Indice de prefixe
    is_ = 0  # Indice de suffixe

    couleure = [0] * n  # 0 : blanc , 1 : gris , 2 : noir

    pi = [-1] * n
    P = [0] * n
    S = [0] * n
    P_star = [0] * n
    S_star = [0] * n

    BLANC , GRIS , NOIR = 0 , 1 , 2

    # --- Procédure récursive interne
    # visiter en profondeur
    def visiter_en_profondeur(u) :
        nonlocal ip , is_

        # Marquer le sommet comme visité (gris)
        couleure[u] = GRIS

        # Enregistrer le sommet dans l'ordre de découverte
        ip += 1
        P[u] = ip
        P_star[ip - 1] = u

        # On decouvre les voisins (extraction du sommet v depuis
           le tuple (v, poids))
        voisins = sorted([v for v, poids in GTL.T[u]])

        # Boucle principale de la procedure
        for v in voisins :
            if couleure[v] == BLANC :
                pi[v] = u
                visiter_en_profondeur(v)

        # Exploration de u est terminer
        couleure[u] = NOIR

        # u est ajouté à l'ordre de fin de visite
        is_ += 1
        S[u] = is_
        S_star[is_ - 1] = u
```

```python

    # --- Boucle principale de DFS
    for u in range(n) :
        if couleure[u] == BLANC :
            visiter_en_profondeur(u)
    return (P , S , P_star , S_star , pi)



# --- 5. Prim algorithme
def prim(graphe_md) :

    n = graphe_md.n
    D = graphe_md.D

    # Initialisations
    cle = [float('inf')] * n
    pi = [-1] * n
    visite = [False] * n

    # Initialiser le tas avec les clés
    F = Tas()
    cle[0] = 0
    F.inserer(0, cle[0])

    while not F.est_vide() :
        # Extraire le sommet u avec la clé minimale
        _ , s = F.extraire_min()

        if visite[s] :
            continue
        visite[s] = True


        # Mettre à jour les clés des voisins de u
        for t in range(n) :
            if s == t :
                continue
            if not visite[t] and D[s][t] < cle[t] :
                cle[t] = D[s][t]
                pi[t] = s
                F.inserer(t, cle[t])
    return pi


# --- 6. Convertir pi en GrapheTL
def pi_vers_graphe_tl(pi) :

    n = len(pi)
```

```
    GTL = GrapheTL(n)

    for v in range(n) :
        u = pi[v]
        if u != -1 :
            GTL.ajouter_arc(u, v, 1)  # Poids par défaut de 1 pour le MST

    return GTL
```

Listing 8 – Code source : utils.py (Prim, Conversion, DFS)

6 Heuristique de la Demi-Somme (HDS)

HDS est un algorithme exact de type Branch Bound. Il explore l'arbre des solutions possibles en utilisant une borne inférieure $h(x)$ robuste. L'implémentation utilise des masques binaires pour optimiser la gestion de l'ensemble des villes visitées.

6.1 Procédures Détaillées

Algorithm 6 Calcul de la Borne Inférieure $h(x)$

Entrée: Graphe G, $Chemin$, $CoutActuel$, $Masque$ (Entier binaire)
Sortie: Borne inférieure estimée

```
Start ← Chemin[0], End ← Chemin[−1]
Somme ← 2 × CoutActuel                    ▷ Les arêtes du chemin comptent double
pour v de 0 à n − 1 faire
  Degre ← 0
                                          ▷ Vérification bit-à-bit si v est visité
  si (Masque ≫ v)&1 alors
    si v == Start OU v == End alors
      Degre ← 1
    sinon
      Degre ← 2
    end si
  end si
  si Degre < 2 alors
    Manque ← 2 − Degre
    Min1 ← ∞, Min2 ← ∞
    pour voisin de 0 à n − 1 faire
      EstVisite ← (Masque ≫ voisin)&1
      EstExtremite ← (voisin == Start OU voisin == End)
                          ▷ Voisin valide s'il n'est pas visité OU est une extrémité
      si NON EstVisite OU EstExtremite alors
        d ← G.D[v][voisin]
        Mettre à jour Min1 et Min2 avec d
      end si
    end pour
    Somme ← Somme + Min1
    si Manque == 2 alors
      Somme ← Somme + Min2
    end si
  end si
end pour
retourner Somme/2
```

Algorithm 7 Algorithme HDS (Branch & Bound)

Entrée: Graphe G
Sortie: Meilleur Chemin Optimal

$MasqueInitial \leftarrow 1 \ll 0$ ▷ *Bit 0 à 1*
$Racine \leftarrow Nud(Ville = 0, Masque = MasqueInitial, Cout = 0)$
$Racine.Borne \leftarrow CalculerBorne(Racine)$
$F \leftarrow Tas()$, $F.inserer(Racine)$
$CoutMin \leftarrow \infty$
tant que NON $F.est_vide()$ **faire**
  $N \leftarrow F.extraire_min()$
  ▷ *Élagage si la borne dépasse déjà le meilleur coût trouvé*
  si $N.Borne \geq CoutMin$ **alors**
    Continuer
  end si
  si $Taille(N.Chemin) == n$ **alors**
    $CoutTotal \leftarrow N.Cout + G.D[N.Ville][0]$
    si $CoutTotal < CoutMin$ **alors**
      $CoutMin \leftarrow CoutTotal$
      $MeilleurChemin \leftarrow N.Chemin$
    end si
  sinon
    pour v de 0 à $n - 1$ **faire**
      ▷ *Si v n'est pas dans le masque des visités*
      si NON $((N.Masque \gg v)\&1)$ **alors**
        $Cout \leftarrow N.Cout + G.D[N.Ville][v]$
        si $Cout < CoutMin$ **alors**
          $NewMasque \leftarrow N.Masque|(1 \ll v)$
          $NewBorne \leftarrow CalculerBorne(...)$
          si $NewBorne < CoutMin$ **alors**
            $Enfant \leftarrow Nud(v, NewMasque, Cout, NewBorne)$
            $F.inserer(Enfant)$
          end si
        end si
      end si
    end pour
  end si
end tant que
retourner $MeilleurChemin$

6.2 Implémentation Python

```python
from structures import NoeudExploration
import numpy as np
import heapq

# --- ALGO 4 :  Heuristique de la Demi-Somme  ---
```

```python
# Implémente l'algorithme HDS pour le problème du TSP en utilisant Branch and Bound.

# Fonction de calcule de la borne inférieure h(x)
def calculer_borne_hds(graphe_md, chemin , cout_actuel , visited_mask):
    n = graphe_md.n
    D = graphe_md.D

    start_noued = chemin[0]
    end_noued = chemin[-1]
    somme_deg = 0

    # 1. Les arêtes pour les villes déjà visitées
    # On ajoute les coûts des arêtes connectant les villes visitées
    # Dans la somme des degres , chaque arret compté deux fois
    somme_deg += 2 * cout_actuel

    # 2. Les arêtes pour les villes non visitées
    for ville in range(n):
        degre_actuel = 0

        # Si la ville est déjà visitée , on passe
        if (visited_mask >> ville) & 1:
            if ville == start_noued or ville == end_noued:
                degre_actuel = 1 # Chaque extrémité du chemin partiel a une seule arête connectée
            else:
                degre_actuel = 2 # Les autres villes visitées ont déjà deux arêtes connectées

        # Si le sommet a déjà deux arêtes connectées , on passe
        if degre_actuel >= 2:
            continue

        # Trouver les arêtes les moins chères pour cette ville
        nb_arret_trouver = 2 - degre_actuel
        min1 , min2 = float('inf') , float('inf')
        for voisin in range(n) :
            if ville == voisin:
                continue

            # Verifier la validite de voisin
            # ca veut dire , verifier que l'arête n'est pas déjà utilisée dans le chemin partiel
            est_visite = (visited_mask >> voisin) & 1
            est_extrémite = (voisin == start_noued or voisin == end_noued)
            if not est_visite or est_extrémite :
```

```
                dist = D[ville][voisin]
                if dist < min1:
                    min2 = min1
                    min1 = dist
                elif dist < min2:
                    min2 = dist

        # Ajouter les arêtes les moins chères trouvées
        if nb_arret_trouver >= 1:
            somme_deg += min1

        if nb_arret_trouver == 2:
            somme_deg += min2

    # Calculer et retourner la borne inférieure
    return somme_deg / 2 ‌‌‌
```

```
    meilleur_chemin = []

    # Variables de securité pour éviter les boucles infinies
    MAX_ITR = 1000000
    nb_itr = 0

    # 2. Exploration de l'arbre
    while tas_priorite :
        # Securite
        nb_itr += 1
        if nb_itr > MAX_ITR :
            print("Alerte : Nombre maximum d'itérations atteint. Arrêt de l'algorithme HDS.")
            break

        # Selectionner le noeud avec la plus petite borne
        noeud = heapq.heappop(tas_priorite)

        # Vérifier si le noeud courant peut mener à une meilleure solution
        # Si la borne est déjà supérieure au coût minimal trouvé, on ignore ce noeud
        # car il ne peut pas conduire à une solution optimale

        if noeud.bound >= cout_minimal:
            continue

        # Vérifier si le chemin est complet
        if len(noeud.path) == n:
            # Fermer le cycle en revenant au noeud de départ
            cout_retour = D[noeud.current_city][start_noued]
            cout_total = noeud.cost + cout_retour

            # si on a trouvé un meilleur chemin
            # Mettre à jour le coût minimal et le meilleur chemin
            if cout_total < cout_minimal:
                cout_minimal = cout_total
                meilleur_chemin = noeud.path
            continue

        # Si le chemin n'est pas complet , on génère les noeuds enfants
        # Brancheemnt ( Separation / Branching )
        # Explorer les villes non visitées
        for ville in range(n) :
            # Vérifier si la ville a déjà été visitée
            if (noeud.visited_mask >> ville) & 1:
                continue

```

```
            # Calcul des nouvelles valeurs pour le noeud enfant
            new_cost = noeud.cost + D[noeud.current_city][ville]
            if new_cost >= cout_minimal:
                continue
            new_path = noeud.path + [ville]
            new_visited_mask = noeud.visited_mask | (1 << ville)
            new_bound = calculer_borne_hds(graphe_md, new_path,
               new_cost, new_visited_mask)

            # Si la nouvelle borne est prometteuse , on ajoute le
               noeud enfant au tas
            if new_bound < cout_minimal:
                enfant = NoeudExploration(current_city=ville,
                                          visited_mask=new_visited_mask,
                                          cost=new_cost,
                                          bound=new_bound,
                                          path=new_path)
                heapq.heappush(tas_priorite, enfant)

    return meilleur_chemin
```

Listing 9 – Code source : hds.py

7 Bilan du Projet : Organisation et Analyse Critique

Cette section détaille la méthodologie adoptée pour mener à bien ce projet, les obstacles techniques surmontés et les pistes envisagées pour aller plus loin.

7.1 Organisation du Travail

Pour garantir un code propre, maintenable et conforme aux exigences, nous avons adopté une approche modulaire et incrémentale :

- **Architecture Modulaire :** Au lieu d'un fichier unique monolithique, le projet a été divisé en packages logiques :
  - `structures/` : Contient les classes de données (GrapheMD, Tas, NoeudExploration).
  - `src/` : Contient l'implémentation des algorithmes et les utilitaires.

  Cette séparation a facilité le débogage et les tests unitaires de chaque composant indépendamment.
- **Développement Incrémental :**
  1. Nous avons commencé par les structures de base (`GrapheMD`) et l'algorithme le plus simple (**PPP**) pour valider la génération de graphes.
  2. Ensuite, nous avons implémenté l'optimisation locale (**OptPPP**) pour voir l'amélioration immédiate.
  3. L'algorithme **HDS**, étant le plus complexe, a été développé en dernier, une fois la structure de Tas et de Nœud validée.

— **Validation par les Tests :** L'étude statistique demandée (100 essais) a servi de validation finale. La cohérence des résultats (HDS toujours optimal, OptPPP proche) a confirmé la justesse des implémentations.

7.2 Difficultés Rencontrées

Plusieurs défis techniques et algorithmiques ont dû être relevés :

1. **Complexité de HDS (Branch & Bound) :** La compréhension et l'implémentation du calcul de la borne inférieure (Heuristique de la Demi-Somme) ont été délicates. Il fallait gérer correctement les arêtes "fixées" par le chemin partiel et les arêtes "libres" pour ne pas sous-estimer ni surestimer la borne, ce qui aurait faussé l'élagage.
2. **Explosion Combinatoire :** Lors des premiers tests de HDS avec $N = 20$, le programme ne se terminait pas. Nous avons dû comprendre que c'était inhérent à la complexité exponentielle du problème et limiter les tests comparatifs à $N = 10$ ou 15 pour avoir des résultats en temps raisonnable.
3. **Gestion de la Récursion (DFS) :** Pour l'algorithme **OptPrim**, le parcours en profondeur sur de grands graphes pouvait atteindre la limite de récursion par défaut de Python. Nous avons résolu cela en augmentant la limite via `sys.setrecursionlimit`.
4. **Visualisation des Données :** Comparer des temps d'exécution allant de 0.1 ms (PPP) à 3000 ms (HDS) sur un même graphique rendait les petites valeurs invisibles. L'utilisation d'une échelle logarithmique (`log scale`) dans Matplotlib a été nécessaire pour rendre les graphiques lisibles.

7.3 Idées d'Amélioration

Si nous avions plus de temps, voici les pistes que nous explorerions pour améliorer le projet :

— **Algorithme de Christofides :** Pour améliorer l'approximation **OptPrim** (qui est une 2-approximation), nous pourrions implémenter l'algorithme de Christofides. Il utilise un couplage parfait de poids minimum sur les sommets de degré impair du MST, garantissant une approximation à 1.5 fois l'optimal.

— **Optimisation de la Borne HDS :** Actuellement, nous utilisons la Demi-Somme des arêtes. Une borne plus forte, comme celle issue de l'algorithme de **Held-Karp** (basée sur la programmation dynamique et les arbres couvrants de poids min avec pénalités), permettrait d'élaguer l'arbre beaucoup plus tôt et de traiter des instances plus grandes ($N = 50+$).

— **Métaheuristiques :** Pour des problèmes de très grande taille ($N = 1000+$), même OptPPP montre ses limites. L'implémentation d'algorithmes génétiques ou de colonies de fourmis donnerait de meilleurs résultats globaux sans le coût prohibitif des méthodes exactes.

7.4 Analyse de Complexité des Algorithmes

Le tableau suivant résume les complexités théoriques des quatre algorithmes implémentés :

Algorithme	**Temps**	**Espace**	**Classe**
PPP	$O(n^3)$	$O(n^2)$	Polynomial (heuristique)
OptPPP	$O(k \cdot n^2)$	$O(n^2)$	Polynomial (amélioration locale)
OptPrim	$O(n^2 \log n)$	$O(n^2)$	Polynomial (approximation)
HDS	$O(b^n)$	$O(n^2 + T)$	Exponentiel (exact)

Table 1 – Synthèse des complexités algorithmiques

Justifications principales :
- **PPP** : n insertions, chacune nécessitant $O(n^2)$ pour trouver le meilleur couple (Q_i, Q_j).
- **OptPPP** : k passes de 2-opt ($k \approx 5 - 10$ en pratique), chaque passe testant $O(n^2)$ échanges.
- **OptPrim** : Prim avec tas binaire en $O(n^2 \log n)$ + DFS en $O(n)$.
- **HDS** : Exploration Branch & Bound avec facteur de branchement $b \approx n - 1$, d'où $O((n-1)^n)$.

8 Étude Statistique et Résultats

Cette section présente l'analyse des performances des quatre algorithmes implémentés. Les tests ont été réalisés sur une machine standard. Nous avons distingué trois phases d'analyse : une étude statistique sur la robustesse ($N = 10$), une étude d'évolution de la complexité (de $N = 5$ à 13) et une démonstration visuelle ($N = 18$).

8.1 Performance et Robustesse (N=10)

Nous avons lancé **100 essais** sur des graphes aléatoires de 10 villes. Voici les moyennes obtenues :

Algorithme	**Coût Moyen**	**Temps Moyen (ms)**	**Écart / Optimal**
PPP	3.1313	0.12	+8.50%
OptPPP	2.9136	0.26	**+0.96%**
OptPrim	3.3983	**0.10**	+17.75%
HDS (Ref)	2.8859	59.62	0.0%

Table 2 – Résultats moyens sur 100 essais ($N = 10$)

Analyse des données :
- **Précision :** L'algorithme **OptPPP** (2-Opt) est extrêmement performant. Il améliore la solution du glouton (PPP) de **6.95%** et parvient à un résultat moyen situé à seulement **0.96%** de la solution optimale fournie par HDS.
- **Vitesse :** Bien que HDS donne la solution parfaite, il est environ **500 fois plus lent** que les heuristiques (59ms contre 0.1ms), même pour un petit graphe de 10 villes.
- **OptPrim :** Sur des données aléatoires uniformes, l'approche par Arbre Couvrant (MST) se révèle moins efficace que les approches géométriques (PPP/OptPPP), bien qu'elle soit très rapide.

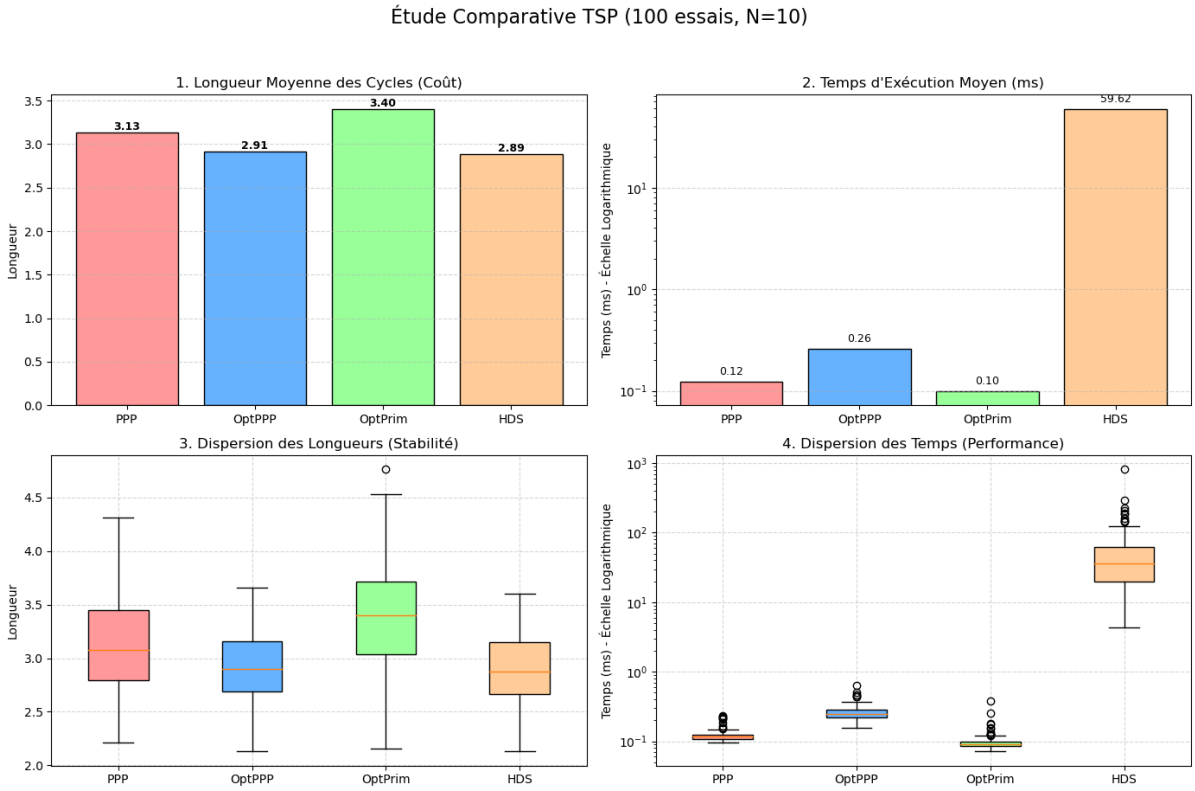

FIGURE 1 – Comparaison statistique : Longueurs (gauche) et Temps Logarithmique (droite)

On observe sur la figure ci-dessus (boîtes à moustaches) que la dispersion des résultats de HDS et OptPPP est très faible, témoignant de leur robustesse, contrairement à OptPrim qui varie davantage selon la topologie du graphe.

8.2 Évolution de la Complexité (N variable)

Pour comprendre le comportement asymptotique, nous avons fait varier le nombre de villes N.

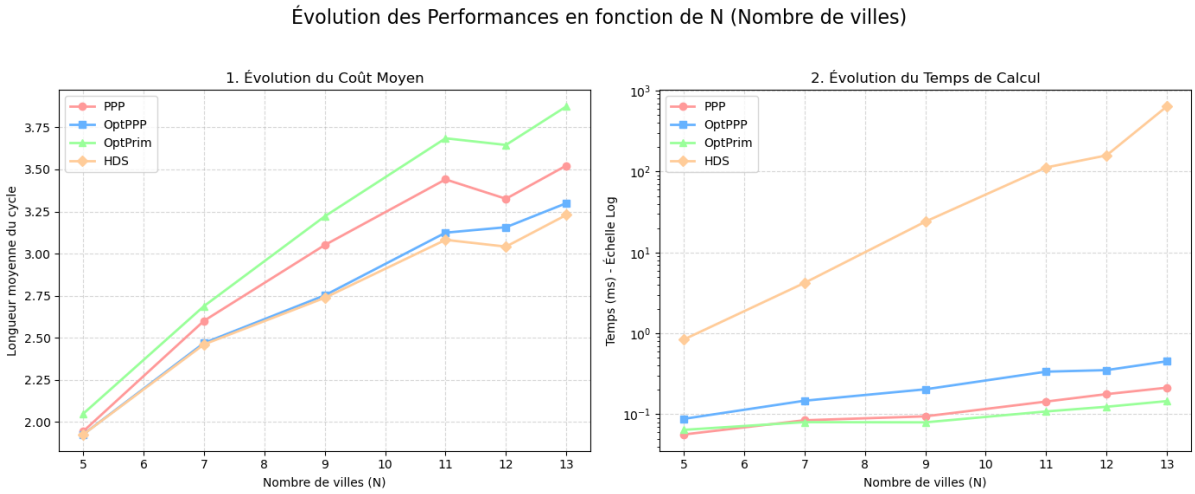

FIGURE 2 – Évolution du Coût et du Temps en fonction de N

Interprétation :

— **Graphique de Gauche (Coût) :** Les courbes de HDS et OptPPP sont quasiment confondues, ce qui confirme que 2-Opt reste une excellente approximation même quand N augmente.

— **Graphique de Droite (Temps) :** L'échelle est logarithmique. La courbe de **HDS** est une ligne droite montante, ce qui illustre visuellement sa **complexité exponentielle** ($O(k^n)$). À l'inverse, les courbes de PPP, OptPPP et OptPrim restent basses et plates, confirmant leur complexité polynomiale. Cela rend HDS inutilisable pour $N > 20$ en temps réel.

8.3 Démonstration Visuelle (Cas N=18)

Pour illustrer concrètement la différence topologique des solutions, voici un exemple d'exécution sur 18 villes.

— **PPP (4.13)** : Le tracé présente des croisements d'arêtes évidents (signe de non-optimalité).

— **OptPPP (3.61)** : L'algorithme a supprimé les croisements ("démêlage"), se rapprochant visuellement de l'optimal.

— **HDS (3.56)** : La solution optimale, légèrement plus courte que OptPPP.

— **OptPrim (4.86)** : Le parcours du MST crée parfois de longs détours, expliquant son coût plus élevé ici.

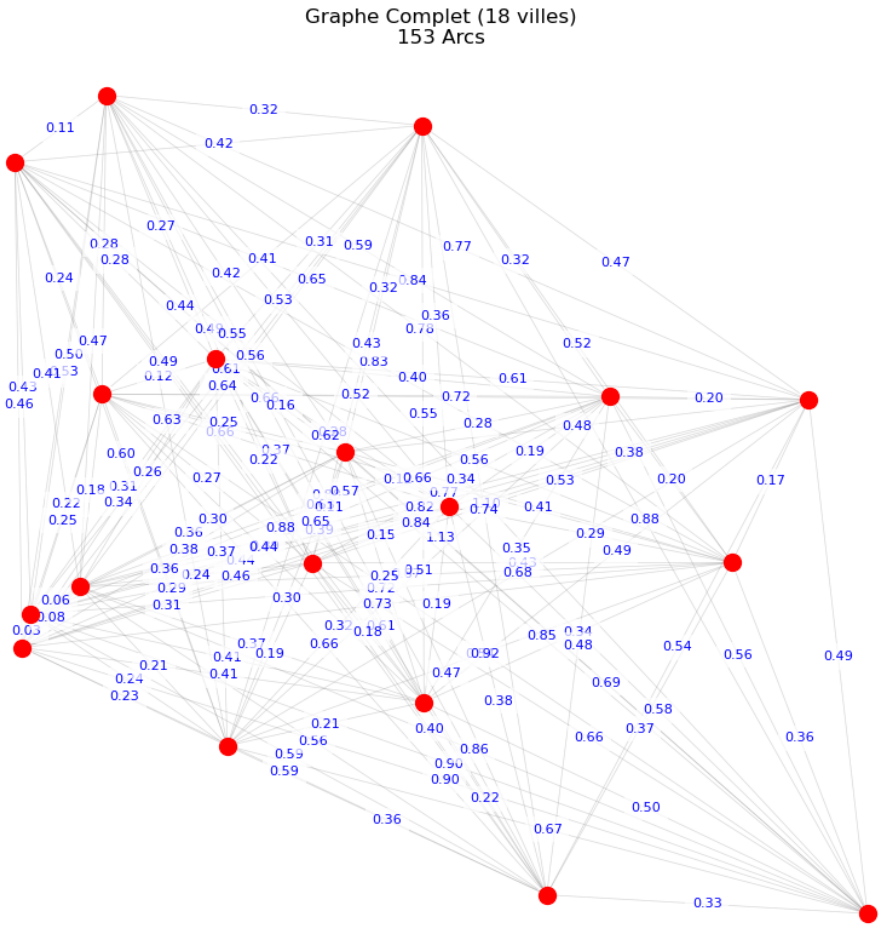

FIGURE 3 – Visualisation de graphe pour N=18

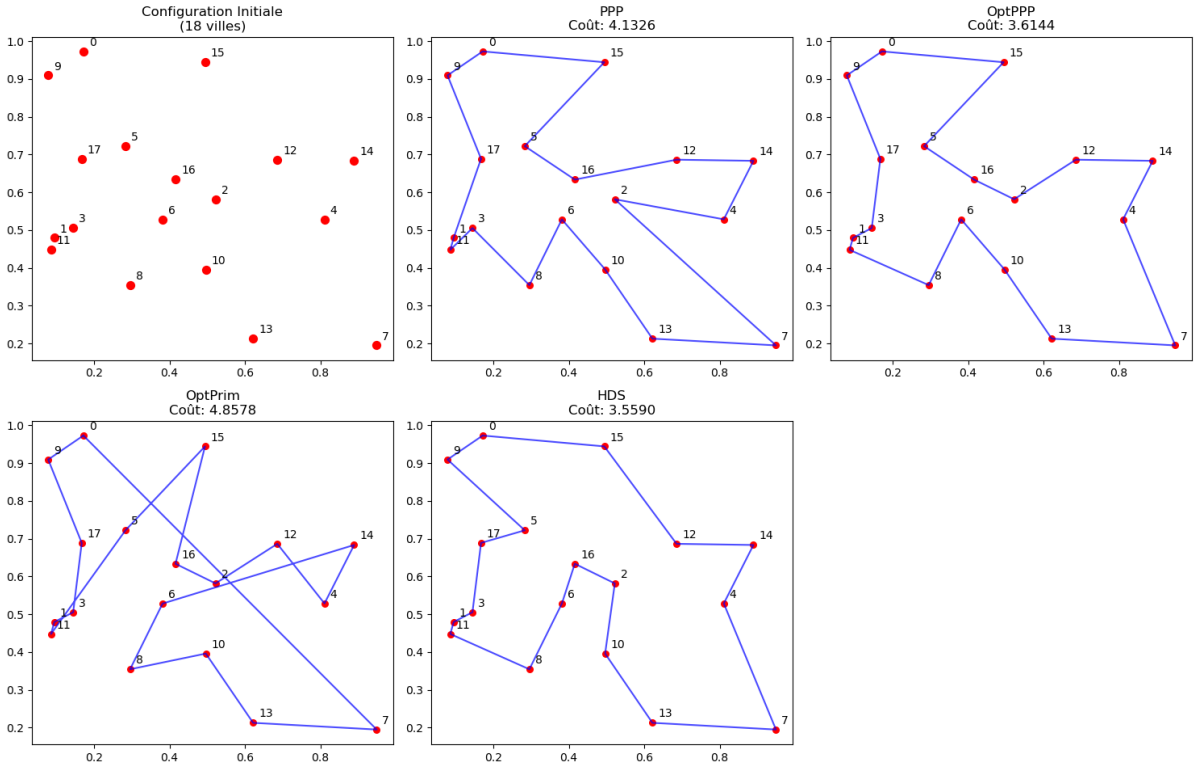

FIGURE 4 – Visualisation des cycles trouvés pour N=18

9 Conclusions

Ce projet nous a permis d'implémenter et de comparer quatre stratégies fondamentalement différentes pour le TSP.

1. **L'Exhaustivité a un prix :** L'algorithme **HDS (Branch & Bound)** est le seul à garantir l'optimalité. Cependant, sa complexité temporelle explose littéralement avec la taille du problème. Il est réservé aux instances critiques de petite taille ($N < 20$).
2. **Le Compromis Idéal :** L'algorithme **OptPPP (2-Opt)** s'impose comme le vainqueur pratique. Il combine la rapidité extrême des heuristiques avec une qualité de solution quasi-optimale (souvent $< 1\%$ d'erreur). Dans un contexte industriel avec des centaines de villes, ce serait le choix par défaut.
3. **Théorie vs Pratique :** L'algorithme **OptPrim** est intéressant pour sa garantie théorique (2-approximation), mais sur des instances euclidiennes aléatoires, il est systématiquement battu par les heuristiques de construction et d'amélioration locale comme OptPPP.

10 Références

10.1 Cours et Documentation Pédagogique

— **C. Selmi** (2025). *Polycopié du cours d'Algorithmique des graphes.* Université de Rouen Normandie.
Disponible sur : `https://universitice.univ-rouen.fr/pluginfile.php/1797227/mod_resource/content/20/Polycopi%C3%A9%20du%20cours%20dAlgorithmique%20des%20graphes.pdf`

10.2 Ressources Algorithmiques

— **Christofides, N.** (1976). *Worst-case analysis of a new heuristic for the travelling salesman problem.* Report 388, Graduate School of Industrial Administration, Carnegie Mellon University.
— **Held, M., & Karp, R. M.** (1962). *A dynamic programming approach to sequencing problems.* Journal of the Society for Industrial and Applied Mathematics, 10(1), 196-210.
— **Prim, R. C.** (1957). *Shortest connection networks and some generalizations.* Bell System Technical Journal, 36(6), 1389-1401.
— **Wikipedia.** *Branch and Bound.*
`https://fr.wikipedia.org/wiki/Branch_and_bound`
— **Gerad.** *Métaheuristique.*
`https://www.gerad.ca/~alainh/Metaheuristiques.pdf`
— **Waterloo University.** *Problème du voyageur de commerce.*
`https://www.math.uwaterloo.ca/tsp/`

10.3 Bibliothèques Python

— **NumPy Documentation.** *NumPy v1.24 Manual.*
`https://numpy.org/doc/stable/`
— **Python Documentation.** *heapq — Heap queue algorithm.*
`https://docs.python.org/3/library/heapq.html`
— **Matplotlib Documentation.** *Matplotlib : Visualization with Python.*
`https://matplotlib.org/stable/index.html`